DATABASE DESIGN & IMPLEMENTATION DOCUMENT

NAILVERSE

Not just NAILS.. IT's the WHOLE MOOD

Online Nail Art & Nail Beauty Online Platform

WEEK - 5

---

5. Payment Transaction Management System

Scenario

Every appointment booked on Nailverse has to be paid for, either online or at the salon. The business therefore needs a reliable record of who paid, how they paid, how much was paid and whether the transaction went through. Sometimes a payment fails and the customer tries again with another method, so the system must keep every attempt, not only the final one.

In this module a PAYMENT table is added to the database. It is linked to the APPOINTMENT table created in Week 4, which in turn is linked to the CUSTOMER table from Week 1. Using this table, the salon can follow each transaction, separate completed payments from failed ones, and prepare revenue and payment-method reports.

1. Create the Payment Table

Payment Entity

Holds one row for every payment attempt made against an appointment.

Attribute	Description
Payment_ID	Unique number given to each payment attempt
Appointment_ID	The appointment this payment belongs to
Payment_Date	Day on which the payment was made
Payment_Mode	UPI / Credit Card / Debit Card / Net Banking / Cash
Payment_Status	Completed / Failed / Pending
Transaction_Amount	Amount paid (in rupees)

```
SQL> CREATE TABLE Payment (
  2       Payment_ID VARCHAR2(7),
  3       Appointment_ID VARCHAR2(7),
  4       Payment_Date DATE,
  5       Payment_Mode VARCHAR2(15),
  6       Payment_Status VARCHAR2(10),
  7       Transaction_Amount NUMBER(10,2),
  8       PRIMARY KEY (Payment_ID),
  9       FOREIGN KEY (Appointment_ID)
 10           REFERENCES Appointment(Appointment_ID)
 11   );

Table created.
```

2. Link Payments to Appointments and Add Constraints

Appointment → Payment

One appointment can have more than one payment record, because a failed attempt may be followed by a successful one. Each payment record belongs to exactly one appointment.

Relationship: APPOINTMENT (1) ——————— (M) PAYMENT

The Appointment_ID column of PAYMENT is a foreign key that refers to Appointment_ID of the APPOINTMENT table. Because every appointment already points to a customer, the customer behind any payment can be found through this link.

The following constraints keep the payment data correct:

- NOT NULL – a payment must always have an appointment and a date.
- CHECK – the mode and status must be one of the allowed values, and the amount must be above zero.

```
SQL> ALTER TABLE Payment MODIFY Appointment_ID CONSTRAINT NN_Pay
_Appt NOT NULL;

Table altered.

SQL> ALTER TABLE Payment MODIFY Payment_Date CONSTRAINT NN_Pay_D
ate NOT NULL;

Table altered.
```

```
SQL> ALTER TABLE Payment
  2  RENAME CONSTRAINT CHK_Pay_Status TO CHK_Pay_Mode;

Table altered.

SQL> ALTER TABLE Payment ADD CONSTRAINT CHK_Pay_Status
  2  CHECK (Payment_Status IN ('COMPLETED','FAILED','PENDING'));

Table altered.

SQL> ALTER TABLE Payment ADD CONSTRAINT CHK_Pay_Amount CHECK (Transaction_Amount > 0);

Table altered.

SQL> DESC Payment;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 PAYMENT_ID                                NOT NULL VARCHAR2(7)
 APPOINTMENT_ID                            NOT NULL VARCHAR2(7)
 PAYMENT_DATE                              NOT NULL DATE
 PAYMENT_MODE                                       VARCHAR2(15)
 PAYMENT_STATUS                                     VARCHAR2(10)
 TRANSACTION_AMOUNT                                 NUMBER(10,2)
```

3. Record Payment Transactions

Payments are added with the INSERT command. The first four records are shown below.

```
SQL> INSERT INTO Payment (Payment_ID, Appointment_ID, Payment_Date, Payment_Mode,
Payment_Status, Transaction_Amount) VALUES ('PAY0002', 'A002', DATE '2026-01-25',
'Credit Card', 'COMPLETED', 1200.00);

1 row created.

SQL> INSERT INTO Payment (Payment_ID, Appointment_ID, Payment_Date, Payment_Mode,
Payment_Status, Transaction_Amount) VALUES ('PAY0003', 'A003', DATE '2026-10-26',
'Credit Card', 'COMPLETED', 1500.00);

1 row created.

SQL> INSERT INTO Payment (Payment_ID, Appointment_ID, Payment_Date, Payment_Mode,
Payment_Status, Transaction_Amount) VALUES ('PAY0004', 'A004', DATE '2026-12-27',
'Credit Card', 'COMPLETED', 800.00);

1 row created.

SQL> INSERT INTO Payment (Payment_ID, Appointment_ID, Payment_Date, Payment_Mode,
Payment_Status, Transaction_Amount) VALUES ('PAY0005', 'A005', DATE '2026-06-28',
'Credit Card', 'COMPLETED', 2000.00);
```

The remaining twelve payments are inserted in the same way. Appointment A006 has two records: the first attempt with a debit card failed, and the customer then paid successfully through UPI.

Payment table after insertion:

```
SQL> SELECT * FROM Payment;

PAYMENT APPOINT PAYMENT_D PAYMENT_MODE    PAYMENT_ST TRANSACTION_AMOUNT
------- ------- --------- --------------- ---------- ------------------
PAY0001 A001    01-SEP-26 Credit Card     COMPLETED                999
PAY0002 A002    25-JAN-26 UPI             COMPLETED               1200
PAY0003 A003    26-OCT-26 Debit Card      FAILED                  1500
PAY0004 A004    27-DEC-26 Net Banking     COMPLETED                800
PAY0005 A005    28-JUN-26 UPI             COMPLETED               2000
PAY0006 A006    29-OCT-26 Debit Card      FAILED                   750
PAY0007 A007    30-MAY-26 UPI             COMPLETED               1800
PAY0008 A008    31-OCT-26 Credit Card     COMPLETED                950
PAY0009 A009    01-FEB-26 Cash            FAILED                  1100
PAY0010 A010    02-APR-26 UPI             COMPLETED               2500

10 rows selected.
```

4. Track Completed and Failed Payments

Tasks: Write SQL queries to:

1. List all payments that were completed.

```
SQL> SELECT * FROM Payment
  2  WHERE Payment_Status = 'COMPLETED'
  3  ORDER BY Payment_ID;

PAYMENT APPOINT PAYMENT_D PAYMENT_MODE    PAYMENT_ST TRANSACTION_AMOUNT
------- ------- --------- --------------- ---------- ------------------
PAY0001 A001    01-SEP-26 Credit Card     COMPLETED                 999
PAY0002 A002    25-JAN-26 UPI             COMPLETED                1200
PAY0004 A004    27-DEC-26 Net Banking     COMPLETED                 800
PAY0005 A005    28-JUN-26 UPI             COMPLETED                2000
PAY0007 A007    30-MAY-26 UPI             COMPLETED                1800
PAY0008 A008    31-OCT-26 Credit Card     COMPLETED                 950
PAY0010 A010    02-APR-26 UPI             COMPLETED                2500

7 rows selected.
```

2. Find the payments that failed.

```
SQL> SELECT * FROM Payment
  2  WHERE Payment_Status = 'FAILED'
  3  ORDER BY Payment_ID;

PAYMENT APPOINT PAYMENT_D PAYMENT_MODE    PAYMENT_ST TRANSACTION_AMOUNT
------- ------- --------- --------------- ---------- ------------------
PAY0003 A003    26-OCT-26 Debit Card      FAILED                   1500
PAY0006 A006    29-OCT-26 Debit Card      FAILED                    750
PAY0009 A009    01-FEB-26 Cash            FAILED                   1100
```

3. Count the payments under each status.

```
SQL> SELECT Payment_Status,
  2         COUNT(Payment_Status) AS Number_of_Payments
  3  FROM Payment
  4  GROUP BY Payment_Status
  5  ORDER BY Number_of_Payments DESC;

PAYMENT_ST NUMBER_OF_PAYMENTS
---------- ------------------
COMPLETED                   7
FAILED                      3
```

5. Study the Payment Methods Chosen by Customers

Prepare reports that show how customers prefer to pay.

Report 1: Payment Method Usage

Find:

- How many transactions were made through UPI.
- How many payments were made with a card (credit or debit).
- The payment method used most often.

```
SQL> SELECT COUNT(8) AS UPI_Transactions
  2  FROM Payment
  3  WHERE Payment_MOde = 'UPI';

UPI_TRANSACTIONS
----------------
               4

SQL> SELECT COUNT(8) AS Card_Payments
  2  FROM Payment
  3  WHERE Payment_Mode IN ('Credit Card', 'Debit Card');

CARD_PAYMENTS
-------------
            ..
SQL> SELECT Payment_Mode, COUNT(*) AS Transaction_Count
  2  FROM Payment
  3  GROUP BY Payment_Mode
  4  ORDER BY Transaction_Count DESC, Payment_Mode;

PAYMENT_MODE    TRANSACTION_COUNT
--------------- -----------------
UPI                             4
Credit Card                     2
Debit Card                      2
Cash                            1
Net Banking                     1
SQL> SELECT *
  2  FROM (
  3      SELECT Payment_Mode, COUNT(*) AS Transaction_Count
  4      FROM Payment
  5      GROUP BY Payment_Mode
  6      ORDER BY COUNT(*) DESC, Payment_Mode
  7  )
  8  WHERE ROWNUM = 1;

PAYMENT_MODE    TRANSACTION_COUNT
--------------- -----------------
UPI                             4
```

Report 2: Revenue Summary

Only completed payments are counted as revenue.

Find:

- The total revenue earned.
- The revenue earned through each payment method.
- The average amount of a transaction.

```
SQL> SELECT SUM(Transaction_Amount) AS Total_Revenue
  2  FROM Payment
  3  WHERE Payment_Status = 'COMPLETED';

TOTAL_REVENUE
-------------
        10249
```

```
SQL> SELECT Payment_MOde, SUM(Transaction_Amount) AS Total_Revenue
  2  FROM Payment
  3  WHERE Payment_Status = 'COMPLETED'
  4  GROUP BY Payment_Mode
  5  ORDER BY Total_Revenue DESC;

PAYMENT_MODE    TOTAL_REVENUE
--------------- -------------
UPI                      7500
Credit Card              1949
Net Banking               800
```

```
SQL> SELECT ROUND(AVG(Transaction_Amount), 2)
  2         AS Average_Transaction_Amount
  3  FROM Payment
  4  WHERE Payment_Status = 'COMPLETED';

AVERAGE_TRANSACTION_AMOUNT
--------------------------
                   1464.14
```

Report 3: Customer-wise Payment Record

Display:

- Customer name
- Appointment ID
- Appointment date
- Payment date
- Payment mode
- Amount
- Payment status

```
SQL> SELECT c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  2         a.Appointment_ID,
  3         a.Appointment_Date,
  4         p.Payment_Date,
  5         p.Payment_Mode,
  6         p.Transaction_Amount AS Amount,
  7         p.Payment_Status
  8  FROM Customer c
  9  JOIN Appointment a
 10      ON c.Customer_ID = a.Customer_ID
 11  JOIN Payment p
 12      ON a.Appointment_ID = p.Appointment_ID
 13  ORDER BY Customer_Name, p.Payment_ID;

CUSTOMER_NAME                          APPOINT APPOINTME PAYMENT_D
-------------------------------------- ------- --------- ---------
PAYMENT_MODE        AMOUNT PAYMENT_ST
------------------- ---------- ----------
ANANYA  SURESH                         A003    26-OCT-26 26-OCT-26
Debit Card            1500 FAILED

HARI KUMAR                             A008    31-OCT-26 31-OCT-26
Credit Card            950 COMPLETED

KAVYA RAO                              A002    25-JAN-26 25-JAN-26
UPI                   1200 COMPLETED


CUSTOMER_NAME                          APPOINT APPOINTME PAYMENT_D
-------------------------------------- ------- --------- ---------
PAYMENT_MODE        AMOUNT PAYMENT_ST
------------------- ---------- ----------
MEERA IYER                             A001    24-OCT-26 01-SEP-26
Credit Card            999 COMPLETED

MOSHAMI JUN                            A007    30-MAY-26 30-MAY-26
UPI                   1800 COMPLETED

NILLA VEERA                            A010    02-APR-26 02-APR-26
UPI                   2500 COMPLETED


CUSTOMER_NAME                          APPOINT APPOINTME PAYMENT_D
-------------------------------------- ------- --------- ---------
PAYMENT_MODE        AMOUNT PAYMENT_ST
------------------- ---------- ----------
NISHA KUMARI                           A005    28-JUN-26 28-JUN-26
UPI                   2000 COMPLETED

PRIYA NAIR                             A004    27-DEC-26 27-DEC-26
Net Banking            800 COMPLETED

SRI VALLI                              A009    01-FEB-26 01-FEB-26
Cash                  1100 FAILED


CUSTOMER_NAME                          APPOINT APPOINTME PAYMENT_D
-------------------------------------- ------- --------- ---------
PAYMENT_MODE        AMOUNT PAYMENT_ST
------------------- ---------- ----------
VARSHA RAJAN                           A006    29-OCT-26 29-OCT-26
Debit Card             750 FAILED


10 rows selected.
```

Payment Method Performance

A combined view of the completed payments under every payment method.

```
SQL> SELECT Payment_Mode,
  2         COUNT(*) AS Transaction_Count,
  3         SUM(Transaction_Amount) AS Total_Revenue,
  4         ROUND(AVG(Transaction_Amount), 2) AS Avg_Transaction
  5  FROM Payment
  6  WHERE Payment_Status = 'COMPLETED'
  7  GROUP BY Payment_Mode
  8  ORDER BY Total_Revenue DESC;

PAYMENT_MODE    TRANSACTION_COUNT TOTAL_REVENUE AVG_TRANSACTION
--------------- ----------------- ------------- ---------------
UPI                             4          7500            1875
Credit Card                     2          1949           974.5
Net Banking                     1           800             800
```